Abby Donis - 22440

Proyecto 3 – Casos de uso LLM

Stardew Bot

Definición del problema y usuario objetivo

Stardew Valley es uno de los juegos más populares de la historia, llegando a un total de 50 millones de copias vendidas en 2026 a lo largo de diversas plataformas, incluidas varias plataformas. Este juego lleva dentro de sí una gran cantidad de información, por ejemplo, los gustos y disgustos de cada uno de los habitantes del pueblo, locación y momentos específicos en los que se pueden encontrar diversos ingredientes y peces, etc.

Memorizar esa enorme cantidad de datos puede ser bastante complicado, cosa que fomentó el uso de una wiki especializada y/o mods que ayuden a los jugadores a llevar un registro de estos datos, sin embargo, esto no es algo posible para aquellos que por alguna razón no pueden acceder a la wiki o instalar mods que les ayuden, además de que la wiki no está hecha para celulares lo que hace que su navegación se dificulte para aquellos en consolas.

El usuario objetivo de este bot es aquel jugador que se encuentra en necesidad de saber algún dato sobre qué regalar o dónde obtener un material sin necesidad de consultar la wiki. El LLM ayuda a que el jugador reciba la información sin tener que realizar diversas búsquedas en la wiki, además de que esta agilidad es útil al considerar que el usuario está jugando en ese momento y no quiere interrumpir su “flow” de juego.

Datos y pipeline del sistema

Consideraciones previas

Las decisiones conforme a la elección de modelos fueron fuertemente influenciadas por las limitaciones del hardware como la falta de una GPU dedicada o espacio suficiente y presupuesto nulo que pudiera utilizar para modelos más avanzados, llevando a buscar modelos que fueran ligeros y gratuitos.

Los datos utilizados para el proyecto provinieron de la wiki más popular de Stardew Valley (https://stardewvalleywiki.com/Stardew_Valley_Wiki), la cual fue creada con herramientas de MediaWiki, permitiendo exportar diversas páginas a un XML. Con este archivo se realizó una limpieza de los datos y transformación a un JSON, que sería utilizado con un RAG para dar la información al modelo. Cabe destacar que aunque la wiki esté disponible en español, la información se ve un tanto limitada al tener ciertas partes con una cobertura de traducción menor.

Este dump tuvo sus desafíos en formato que se tenían que manejar en el preprocesamiento: el texto de la wiki tiene plantillas anidadas, tablas, elementos de HTML varios, imágenes y enlaces internos. Varios datos que eran bastante valiosos para el proyecto estaban contenidos dentro de un campo en un elemento llamado “infoboxes” por lo que se debía tener una especial consideración de estos. Además de esto algunas páginas, al ser exportadas tenían “_” en lugar de espacios por lo que sería necesario normalizar estos. Al ser una wiki se debe considerar que hay que realizar este proceso de

exportación de páginas y preprocesamiento de datos para actualizar la información, por fortuna las actualizaciones al juego no son tan comunes por lo que no es tan común tener que repetir este proceso.

Preprocesamiento

El pipeline del proyecto comienza en 'cleanup.py', aquí el XML pasa por varias etapas para poder darle uso en RAG.

Primero hay que lidiar con los diversos elementos de formato que hay, las plantillas que la wiki usa para listar sus elementos "{{Name|x}}" se expanden al argumento x, ósea que el resultado sería solamente "x", así obteniendo las listas de ítems. Las tablas con formato "{|...|}" son pasadas a texto simple siguiendo un formato de "Item: *Nombre*. *Header tabla* is *valores*" así conservando la información de regalos, recetas y lugares, las tablas de horarios son descartadas por ser consideradas ruido en este caso. Antes de eliminar plantillas de las paginas se extraen los elementos de las infoboxes, sobre todo por contener los mejores regalos para los personajes y se hace un chunk de estas y no se mezclan con la info genérica de la página sobre amistad de la wiki.

Ahora lo que sigue es el chunking de la información, gracias a que MediaWiki divide por encabezados las secciones de información, los cuales se van a convertir en chunks (puede ser 1 o más). Las páginas técnicas (como Mining, Fishing, Crops, etc.) los chunks son de máximo 800 caracteres sin overlap mientras que al resto se le agrega un overlap de 100 con el motivo de mantener contexto. Estos chunks tienen metadatos, llevando "{page, section, text}", este último recibe un prefijo con formato "<title> - <section>: " así el retriever tenga señal de contexto incluso en chunks que sean pequeños (aunque los que tienen menos de 20 caracteres se descartan ya que no tienen mucha información).

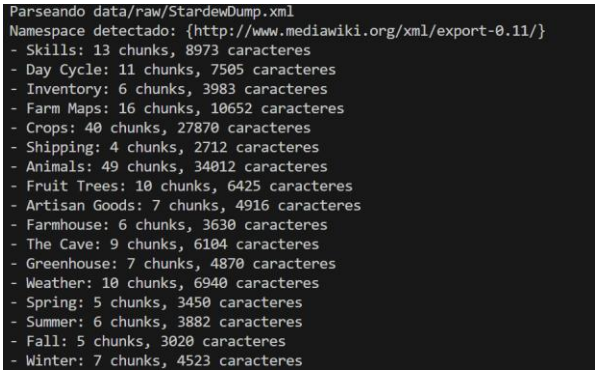


Imagen 1. Fragmento de 'cleanup.py' ejecutándose y obteniendo la información

RAG y búsqueda semántica

Se hizo uso de RAG con prompting few-shot sin fine-tuning, esto debido a que la información puede cambiar con cada actualización del juego y esto permite que se pueda reindexar la información sin tener que volver a entrenar. Es de transfer learning a escala (modelo pre-entrenado y conocimiento inyectado en inferencia), los embeddings toman el papel de espacio latente compartido (la similitud coseno). El fine-tuning no se realizó debido al costo computacional y volatidad de cambio (por ser una wiki), aunque en un equipo más avanzado se podría probar mejores modelos en un futuro.

Los modelos y recursos seleccionados para el proyecto (con las limitaciones ya mencionadas anteriormente) fueron los siguientes por ser de costo nulo.

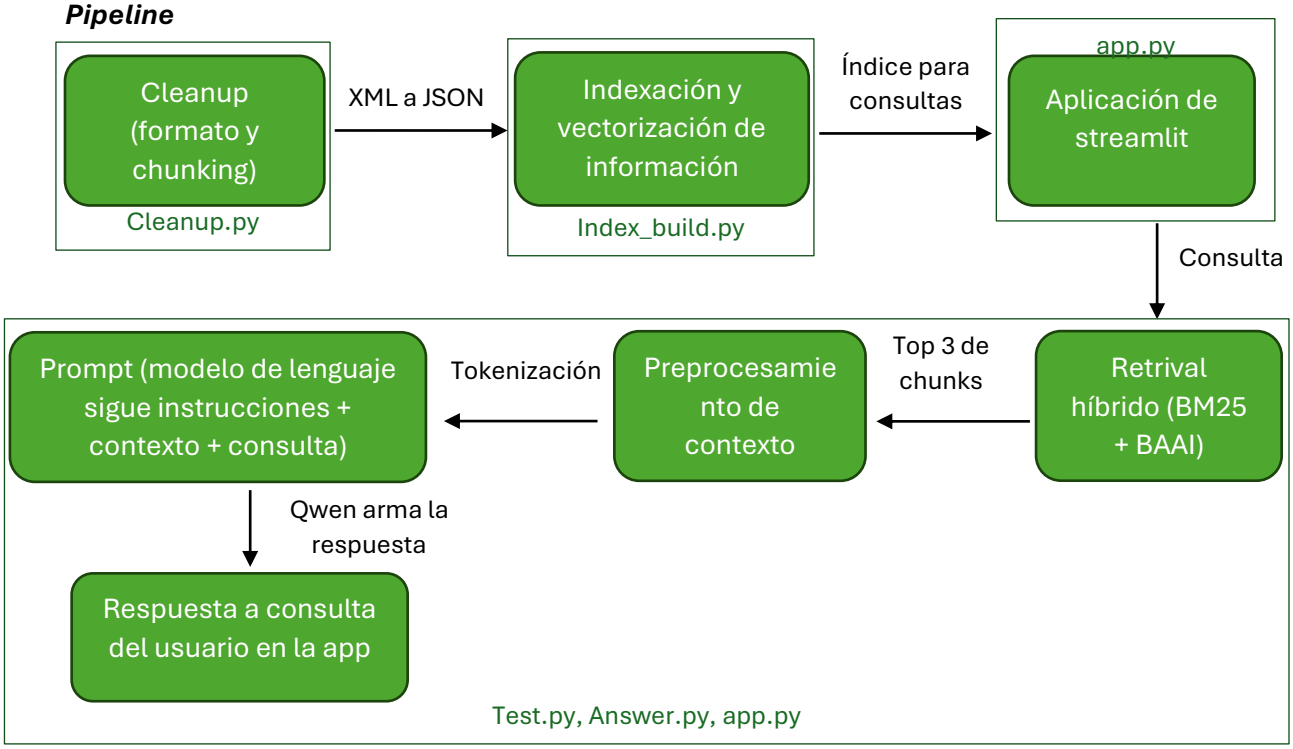
- BM25: Algoritmo de recuperación de información que calcula la relevancia de cierta información a la búsqueda que se hace. Este modelo es estándar en varios sistemas de búsqueda y bases de datos, siendo preferido por su eficacia en encontrar palabras clave específicas.
- BAAI/bge-small-en-v1.5: Modelo open source que genera embeddings de texto, está basado en BERT y es bastante ligero con tan solo 33.4 millones de parámetros lo que aumenta la eficiencia del modelo.
- Qwen2.5-0.5B-Instruct: Modelo open source de lenguaje con 490 millones de parámetros (bastante bueno para computadoras con recursos limitados) que específicamente sigue las instrucciones del usuario y responder a lo que este le escriba de forma directa. Además, funciona muy bien con JSON. Sin embargo, debido a las limitaciones de memoria (cosa que llevó al descarte de modelos Qwen más avanzados) este sufre de una tasa mayor de falsos rechazos y una calidad de redacción menor.
- ChromaDB: Base de datos vectorial open source que tiene la función de facilitar el trabajo con LLMs, almacena vectores y metadatos, realiza búsqueda semántica en la base de datos e implementa sistemas RAG (conecta el modelo de lenguaje con los datos propios que tenemos para que use estos mismos para dar sus respuestas).

Componentes	Modelos	¿Porqué?
Embeddings	BAAI/bge-small-en-v1.5	Modelo gratuito y ocupa poco espacio, además es rápido en CPU Las dimensiones de vector son de 384 lo que significa un balance para costo computacional con expresividad
Lenguaje	Qwen2.5-0.5B-Instruct	
Métrica de similitud	Coseno (ChromaDB)	Estabiliza el ranking de chunks de distintos tamaños
Base vectorial	ChromDB	No necesita un servidor y es capaz de correr en CPU
Búsqueda	Modo híbrido (BM25 en búsqueda léxica y embeddings en búsqueda semántica)	La mezcla de estos hace que se puedan encontrar términos específicos y paráfrasis. Se controla el balance entre estos por medio de “Alpha” (0.3) que hace que la búsqueda semántica sea mayoritaria.
Otros	Boos de entidad de 0.05, penalización para chunks genéricos de -0.05 y uso de stopwords y top-k de 3	Estos ayudan a traer balance y evitar que haya chunks que se “coman” a otros.

Tabla 1. Componentes, sus modelos y recursos seleccionados

La búsqueda se volvió híbrida por como se ayudan entre sí, BM25 encuentra términos exactos que pueden pasar desapercibidos en embedder en chunks que sean bastante grandes. Por otro lado los embeddings encuentran los sinónimos y paráfrasis que BM25 no, así pudiendo usar el "contexto" para dar una respuesta. ChromaDB guarda el texto vectorizado y además ofrece persistencia en disco y metadatos filtrables y es más simple.

Pipeline



Aproximadamente se hace uso de 985 tokens por consulta (dividiendo en 865 para entrada, ósea prompt más el contexto y más o menos 120 para las salidas), la mayor cantidad proveniente de los chunks usados en el contexto.

Evaluación y medición de respuestas

Los 50 casos de prueba en "eval_set.json" fueron construidos con un query especifico, la página de la que se espera obtener la información, sección de la que se espera, palabras clave esperadas y casos en los que se espera que se dé un rechazo a responder el query en cuestión.

En nuestro caso contamos con querys de las siguientes "clases": regalos para cada personaje, peces, cultivos, minerales/minería, museo, armas, ubicaciones, ítems especiales y casos de rechazo. Estos se validaron con conocimiento manualmente revisado en la wiki y mi propio conocimiento como fan de este juego.

Métricas utilizadas

- Recall@3: Fracción de casos en la que al menos uno de los 3 chunks recuperados pertenece a la página y sección que se esperan.

- MRR (Mean Reciprocal Rank): “Ranking” de chunks según la posición en la que el chunk correcto está. Si este está en la posición 1, suma 1. Si está en la posición 2, suma 0.5; si está en la posición 3, suma 0.33 y en caso no aparece, suma 0.



Imagen 2. Resultados de evaluación

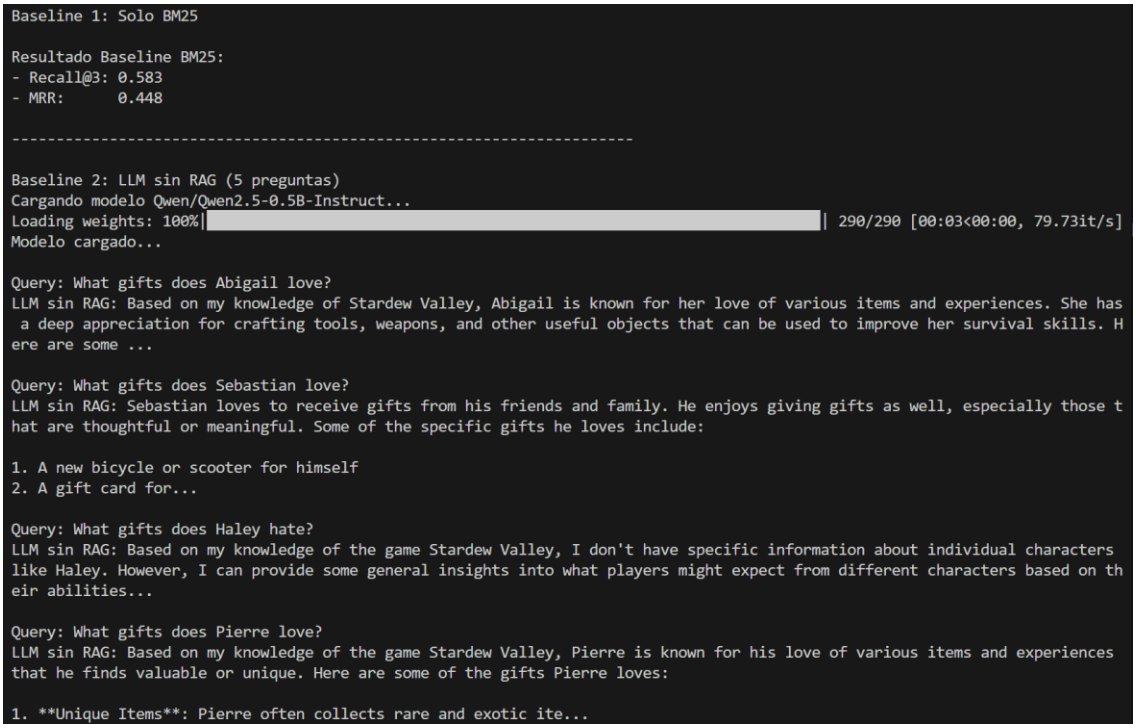


Imagen 3. Resultados baseline

Conjunto de evaluación			
Alpha (α)	**recall@3**	**MRR**	**Fallos**
0.3	0.917	0.813	4
0.5	0.833	0.715	8
0.7	0.771	0.625	11
1.0	0.562	0.451	21

Baseline

Sistema	recall@3	MRR
Solo BM25	0.583	0.448
Comparación RAG	0.917	0.813

Tabla 2. Resultados métricas

Los resultados indican que nuestros mejores resultados se obtuvieron al usar Alpha 0.3, indicando que el mejor balance entre BM25 y los embeddings es favoreciendo a este último, cosa que puede explicarse al ver la cantidad de chunks con las que hay que lidiar. Siguiendo la fórmula “score = α·BM25 + (1−α)·cos” . Los fallos que podemos ver en este se deben a que algunas páginas que estaban más escondidas pasaron por alto en el XML o que recaen en ser obstruidas por chunks con contenido genérico (cosa que provoca la limitación de BAAI).

El baseline se evaluó para ambos métodos de búsqueda por separado. En las pruebas de BM25 por su cuenta podemos notar que hay mejoras bastante grandes al usar el RAG híbrido, este último superando a recall@3 y MRR por 0.334 y 0.365 respectivamente. Por otro lado, tenemos una prueba del LLM sin RAG, lo cual genera alucinaciones en todas y cada una de sus respuestas, por ejemplo, evadiendo de cierta forma responder, anunciando que hay personajes que dan regalos a menudo (cosa que no es cierta) o comparando “rasgos” que estos personajes no tienen. Así demostrando que es mejor combinar ambas para aprovechar lo mejor de ambas.

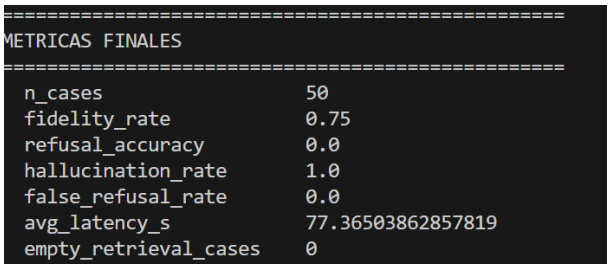


Imagen 3. Métricas RAG

Las métricas utilizadas en RAG son las siguientes:

- Fidelity rate: Fracción de respuestas que si tuvieron mínimo una de las palabras clave/información que se esperaba.
- Refusal accuracy: cantidad de los casos de rechazo correctamente manejados.
- Hallucination rate: Fracción de los casos de rechazo en los que el modelo no respondió la respuesta con lo indicado para esos casos y en efecto, alucinó.
- False refusal rate: Respuestas incorrectas cuando el modelo alucina.
- Avg latency s: Tiempo en segundos de la latencia.
- Empty retrieval cases: Casos en los que retriever no trajo trajo ningún chunk.

Los resultados que vemos aquí se lo podemos atribuir tanto a las limitantes de modelo como a algún chunk que podría haber quedado truncado.

Además, podemos ver que la latencia es de aproximadamente 77.4 segundos cosa bastante alta sin embargo explicable. El tamaño del prompt que le dimos al modelo es un tanto grande y una inferencia en CPU. Se logra mantener un costo de 0 en toda la ejecución del pipeline.

Iteraciones

Se realizó un total de 7 iteraciones, en las cuales se obtuvieron los siguientes resultados

#	Cambios	Recall@3	MRR	Fidelidad	Alucin.	Notas
1	RAG base, BM25, chunking sin overlap, prompt y Qwen2.5-1.5B	0.52	0.47	---	---	Fallo en memoria por modelo de lenguaje, no cabía en RAM
2	Cambio a Qwen2.5-0.5B	0.52	0.47	---	---	Modelo más pequeño que si cupo
3	Correcciones en error de importación en nombres de páginas de wiki y correcciones en eval_set.json.	0.76	0.67	---	---	Ya no se cuentan casos de refuse en el recall y MRR. Me confundí en unos queries de eval_set.
4	Se añadieron stopwords en el tokenizador	0.5	0.42	---	---	Accidentalmente se me olvido poner la sección de regalos bien
5	Regeneración de XML al no haber importado otros algunas cosas por accidente	0.58	0.47	---	---	---
6	Arreglo en cleanup de procesamiento de tablas	0.58	---	---	---	Extracción información del infobox y corrección en procesamiento de tablas, se reindexó
7	Arreglos finales en cleanup, corrección final en eval_set	0.917	0.813	0.75	1.0	Arreglos finales en lectura de tablas y uso de regex. Corrección de nombres de páginas en eval_set. 0.5B no respeta el refusal

Tabla 3. Iteraciones

La mayoría de las mejoras provinieron de arreglar cosas en el cleanup (preprocesamiento) de los datos y corregir accidentes propios en la obtención de datos y eval_set.

Producto final

La interfaz de la aplicación hace uso de colores verdes y un poco más naturales debido a la temática del juego, incluyendo una pequeña referencia a los junimo (que ayudan bastante al jugador en cierto punto del juego) y se muestra la “fuente” o pues los chunks de los que salió la información con su puntuación respectiva junto con su “score”.

Para usarla el usuario simplemente debe escribir su pregunta en inglés, ya que los datos están en ese idioma, esperar (bastante tiempo en mi caso por las limitaciones) y ver la respuesta e incluso expandir la lista de fuentes.



Imagen 4. Aplicación de Stardew Bot y fragmento de resultados de búsqueda de regalos

En caso de que no hay respuesta hay una frase específica para responder y no dejar que simplemente muera.

Limitaciones, riesgos y consideraciones éticas

Limitaciones

- El modelo de Qwen seleccionado es algo limitado por lo que puede llegar a alucinar cuando se dan casos de refuse. Esto se podría corregir con un modelo un poco más avanzado.
- El contexto era bastante extenso en varias áreas los chunks son considerablemente pequeños al poder manejar como máximo 800 caracteres debido a la limitación al trabajar sin GPU.
- La latencia fue de aproximadamente 77 segundos, bastante alta pero puede explicarse por limitaciones del equipo o incluso se podría intentar bajar el número de caracteres máximos para las respuestas.
- La wiki no tiene tanta cobertura de datos en ingles como español, creando una barrera de idioma (aunque se podría traducir la respuesta y pregunta con otras herramientas)

Riesgos

- No se almacena nada de lo que el usuario pregunte por lo que no hay riesgo de privacidad alguno
- El modelo Qwen es vulnerable ante el prompt injection debido a su tamaño tan simple.
- No hay riesgos de que haya generación de contenido inapropiado o dañino gracias a los limites de la información proporcionada.

Cuando no usar

- No usar si no se tiene cierto conocimiento de inglés, ya que los datos y repuestas estarán en este idioma
- No es buena idea usarse para desafíos como speedruns ya que funciona a partir de una wiki, no una guía.
- Si la wiki no ha recibido un XML con datos nuevos que puedan haber en nuevas actualizaciones del juego.

Conclusiones

- Retrieval funciona bastante bien, y a pesar de que hay una fidelidad del 75% la generación por parte de Qwen es limitada debido al modelo utilizado sobre todo en casos de refusal.
- La latencia es grande, pero únicamente por limitaciones de hardware es aceptable.
- Se debe considerar a futuro el implementar modelos más avanzados y probar el proyecto en una máquina con GPU. Además de esto conviene implementar el uso de varios idiomas.
- Un caché de respuestas podría ayudar a reducir la alta latencia.
- Sería bueno revisar aún mas detenidamente cada elemento de la wiki y evaluar el cleanup para poder corregir lo necesario en caso el formato siga irrumpiendo en los elementos.

El proyecto es funcional y gratuito, demostrando que a pesar de limitaciones fuera de mi control es posible crear un pipeline RAG es alcanzable sin el uso de un GPU. El sistema es capaz de recuperar aproximadamente el 91.7% de los casos y genera respuestas fieles en un 75%. Las limitaciones encontradas son consecuencias de varias áreas por lo que es recomendable estudiar los procesos y modelos que se podría usar en condiciones más ideales.

Git https://github.com/abbydoag/Proyecto3_LLM

Fuentes

BAAI/BGE-Small-en-V1.5 · Hugging face. (s. f.). https://huggingface.co/BAAI/bge-small-en-v1.5

Qwen/Qwen2.5-0.5B-Instruct · Hugging face. (2001, 2 marzo). https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

Hugging Face. (s. f.). *Qwen2.5-0.5B-Instruct*. Recuperado 5 de noviembre de 2026, de https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

K, H. (2025, 7 septiembre). *BM25: The Search Algorithm Powering Modern AI Systems*. Medium. https://medium.com/data-science-collective/bm25-the-search-algorithm-powering-modern-ai-systems-b7da2df86665

Ricadela, A. (2025, 15 abril). *What is Chroma? An open source embedded database*. https://www.oracle.com/database/vector-database/chromadb/